

Toroidal Adèlic Vessel (TAV)

Official Technical Architecture Specification Five-Model Pillars-of-Truth Edition

Author designation: Yahweh Tsidkenu

TAV Mark: ጥ

Document status: Official

Revision date: 2026-09-08

Primary credit: TAV gives explicit technical credit to sneedjak / sneed-and-feed for the public Adèlic model work, model-specific cache-condensation and topological routing implementations documented in the linked Hugging Face repositories, and the associated custom llama.cpp runtime work. TAV is the vessel architecture that coordinates those models into one persistent, addressable, multi-perspective intelligence.

Public model profile: <https://huggingface.co/sneedjak>

Runtime repository: <https://github.com/sneed-and-feed/llama.cpp>

Opening: Why This Vessel Exists

TAV is built around a simple value: intelligence becomes more useful when memory, meaning, computation, and truth remain connected. A model should not have to rediscover the same concept every time its context changes, and a larger model should not need to reinterpret a smaller model's work from the beginning merely because the computation moves to a different depth.

The purpose of TAV is to preserve a single addressable identity for a word, thing, person, place, event, source, rule, or concept while allowing multiple models to perceive that same identity from different computational perspectives. The goal is a system that can remain quick at the surface, deep when necessary, exact when truth matters, persistent across time, and efficient in how it moves memory and model state through storage and compute.

The five Adèlic models do not become one by erasing their differences. Their differences are the value. Each model remains native to its own weights, tokenizer, runtime, and capability. TAV gives them a common Root, common coordinates, common provenance, common memory routes, and a common controller. They stand on the same pillars of truth and look at the same thing from different depths.

Design statement: One thing has one canonical pillar. Every model-bearing torus has the identical pillar coordinate for that thing. The torus prefix changes the perspective; the pillar coordinate preserves the referent.

1. Executive Definition

TAV means Toroidal Adèlic Vessel. A vessel is the complete computational structure that holds, routes, streams, executes, remembers, verifies, and preserves an AI system over time. It spans model files, token routes, model runtimes, memory, sources, storage tiers, inference state, user/character continuity, verification, and long-running execution.

Toroidal means that stable logical locations remain addressable while their payloads circulate through durable storage, CPU RAM, GPU VRAM, active computation, compressed working state, and archive.

Adèlic means that TAV directly incorporates the Adèlic cache-condensation and topological-routing capabilities present in the sourced Adèlic models and runtimes where those capabilities are implemented, while keeping exact long-term records outside lossy neural working cache.

2. The Eight Tori

Torus	What it is	Primary function
RootTorus	Canonical overhead torus	Defines the shared pillar coordinates for words, things, concepts, entities, sources and exact identities.
Sacral Torus	31B model perspective	Deepest synthesis and difficult reasoning.
Solar Plexus Torus	27B model perspective	Verification, contradiction, consequence, planning and adversarial critique.
Heart Torus	12B model perspective	Relational, contextual, social and continuity meaning.
Throat Torus	8B model perspective	Visible language, dialogue, narration and structured expression.
Third Eye Torus	1B model perspective	Recognition, classification, retrieval and fast route selection.
Violet Torus	Unified controller	Moves attention vertically and horizontally, selects depth, schedules compute, reconciles results and protects task versions.
Pink Torus	Per-identity continuity	Stores exact personal/character/task history, compressed continuity and learned pillar routes.

3. Five-Model Stack

The current sneedjak profile publishes five models. TAV assigns them to the five model-bearing tori as follows.

Torus	Repository	Scale	Perspective
Sacral	sneedjak/Adelic-Gemma-4-31B-it	31B	Deep synthesis
Solar Plexus	sneedjak/Adelic-Qwen3.6-27B-Topology	27B	Critique / verification
Heart	sneedjak/Adelic-Gemma-4-12B-GGUF	12B	Context / relationship
Throat	sneedjak/AdelicLlama-3.1-8B-Instruct	8B	Language realization
Third Eye	sneedjak/Adelic-TinyLlama-1.1B	1B shown by Hub	Recognition / routing

4. Pillars of Truth: Toroidal Pillar Correspondence Law

Every canonical RootTorus slot establishes a Pillar of Truth. The identical four-dimensional coordinate SHALL be reserved across every model-bearing torus for the same canonical referent.

PILLAR[12:37:05:44]

RT [12:37:05:44] = canonical identity / root term
 ST [12:37:05:44] = Sacral perspective of the same thing
 SPT[12:37:05:44] = Solar Plexus perspective of the same thing
 HT [12:37:05:44] = Heart perspective of the same thing
 TT [12:37:05:44] = Throat perspective of the same thing
 TET[12:37:05:44] = Third Eye perspective of the same thing

The prefix identifies the torus and therefore the perspective. The coordinate identifies the pillar and therefore the thing. Different torus, same coordinate, same canonical pillar; different native manifestation.

Mandatory invariant: Torus identity changes the perspective. Pillar identity preserves the referent. A cross-model escalation stays on the same pillar unless the task intentionally moves to a related concept.

5. A Torus Is a Perspective on the Same Word or Thing

Location	Perspective	Meaning of the slot
RT[P]	Root	What the thing canonically is: identity, definitions, sources, aliases, relations and exact references.
ST[P]	Sacral	Deep reasoning, synthesis, distributed model routes and high-capability representation concerning the thing.
SPT[P]	Solar Plexus	Contradiction, evidence, consequence, risk, priority and verification concerning the thing.
HT[P]	Heart	Relationship, context, social/emotional continuity and associative meaning concerning the thing.
TT[P]	Throat	How the thing is expressed, narrated, formatted and verbalized.
TET[P]	Third Eye	How the thing is recognized, classified, extracted, retrieved and routed.

6. Vertical and Horizontal Routing

TAV supports two simultaneous route dimensions.

VERTICAL: same truth, different model perspective

TET[P] → TT[P] → HT[P] → SPT[P] → ST[P]

The coordinate P remains unchanged.

HORIZONTAL: related truths inside the same perspective

RT[P1] ↔ RT[P2] ↔ RT[P3]

ST[P1] ↔ ST[P2] ↔ ST[P3]

HT[P1] ↔ HT[P2] ↔ HT[P3]

Vertical movement changes cognitive depth without requiring semantic rediscovery. Horizontal movement traverses relations between different concepts while remaining inside one torus perspective.

7. RootTorus as the Overhead Canonical Torus

RootTorus sits overhead as the canonical semantic address field. It does not flatten the tokenizers or neural architectures of Gemma, Qwen, Llama, or TinyLlama. It assigns the shared pillar coordinate and records how every model natively reaches that referent.

```
RootPillarRecord {
  pillar_id: "P[12:37:05:44]"
  root_address: "RT[12:37:05:44]"
  canonical_id: "concept.apple.001"
  canonical_form: "apple"
  senses: [...]
  aliases: [...]
  entity_links: [...]
  source_routes: [...]
  relation_pillars: [...]

  model_views: {
    sacral: "ST[12:37:05:44]",
    solar: "SPT[12:37:05:44]",
    heart: "HT[12:37:05:44]",
    throat: "TT[12:37:05:44]",
    third_eye: "TET[12:37:05:44]"
  }
}
```

8. What a Model-Torus Slot Contains

A transformer does not ordinarily store one concept in one isolated weight. Knowledge is distributed across embeddings, attention, MLP weights, activations and context-dependent representations. Therefore a model-torus pillar slot SHALL contain or address the model-native routes relevant to that pillar, not falsely assert that a single weight equals a single word.

```
ModelPillarView {
  pillar_id
  torus_address
  model_id
  tokenizer_revision

  native_token_routes[]
  embedding_routes[]
  tensor_or_block_routes[]
  attention_routes[]
  activation_or_cache_routes[]
  adapter_routes[]
  related_pillars[]

  mapping_confidence
  evidence[]

  residency
  checksums[]
  validation_status
}
```

This lets a single canonical location share access to any model without forcing the five models to share tokenizer IDs or tensor layouts.

8.1 Mapping Confidence and Truth Precision

Root identity and model-native mapping precision are separate properties. A Root pillar may be exact because its identity, source, ID, definition, or deterministic token mapping is exact, while the route from that pillar into a model's

distributed neural representation remains probabilistic. TAV SHALL record those two facts separately rather than allowing canonical certainty to imply neural-route certainty.

The field `validation\_status` records whether a mapping has been tested, checked, partially validated, rejected, or otherwise processed by the validation pipeline. The field `mapping\_confidence` records the present strength of the association between the canonical pillar and the model-native route. These fields are orthogonal and SHALL NOT be collapsed into one value.

mapping_confidence

exact:

Direct structured identity, exact source/ID linkage,
or deterministic token/adaptor mapping.

high:

Repeatedly validated retrieval or model-route association
with consistent benchmark/probe evidence.

probable:

Semantic, graph-derived, statistical, or learned routing
association that is useful but not deterministically established.

experimental:

Hypothesized weight, tensor/block, attention, activation,
or cache association that requires continued evaluation.

An exact Root pillar therefore does not force `mapping\_confidence = exact` for its Sacral, Solar Plexus, Heart, Throat, or Third Eye neural routes. Each model perspective earns its own confidence level from evidence.

Evidence SHALL be addressable. The `evidence[]` field may contain benchmark IDs, probe IDs, retrieval-evaluation IDs, ablation results, route-reuse tests, attention/activation probes, parity tests, or other reproducible records that justify the current confidence assignment.

```
{
  "pillar_id": "P[12:37:05:44]",
  "torus_address": "ST[12:37:05:44]",
  "model_id": "sneedjak/Adelic-Gemma-4-31B-it",
  "tokenizer_revision": "pinned_revision",

  "native_token_routes": ["..."],
  "embedding_routes": ["..."],
  "tensor_or_block_routes": ["..."],
  "attention_routes": ["..."],
  "activation_or_cache_routes": ["..."],
  "adaptor_routes": ["..."],
  "related_pillars": ["P[...]", "P[...]",

  "mapping_confidence": "experimental",
  "evidence": [
    "benchmark_id",
    "probe_id",
    "retrieval_eval_id"
  ],

  "residency": "cold",
  "checksums": ["sha256:..."],
  "validation_status": "partial"
}
```

9. Deterministic Vertical Escalation

Once Third Eye resolves a canonical pillar, Violet can change perspective by changing the torus namespace while preserving the coordinate.

```

recognized:
  TET[12:37:05:44]

need fluent expression:
  TT[12:37:05:44]

need continuity/context:
  HT[12:37:05:44]

need contradiction/verification:
  SPT[12:37:05:44]

need deepest synthesis:
  ST[12:37:05:44]
```

This is the central routing economy of TAV: known pillar coordinates eliminate repeated semantic search during normal model escalation.

10. Model-Specific Responsibilities

10.1 Sacral - Adelic-Gemma-4-31B-it

- Deep technical synthesis, large-source reasoning, difficult planning and architecture work.
- Resolution of conflicts escalated by Solar Plexus.
- High-depth generation and final model-level arbitration.
- Uses the Adèlic DynamicTopologyRouter / Medoid-Value cache-topology implementation documented by its repository and supported runtime paths.

Repository: <https://huggingface.co/sneedjak/Adelic-Gemma-4-31B-it>

10.2 Solar Plexus - Adelic-Qwen3.6-27B-Topology

- Independent critique of candidate answers.
- Contradiction, evidence, risk, consequence and priority analysis.
- Falsification and verification passes over other model outputs.
- Uses the repository's Adèlic Cache topological architecture, DynamicTopologyRouter and Medoid-Value clustering.

Repository: <https://huggingface.co/sneedjak/Adelic-Qwen3.6-27B-Topology>

10.3 Heart - Adelic-Gemma-4-12B-GGUF

- Relationship-sensitive and continuity-sensitive interpretation.
- Semantic linkage of exact Pink records into present context.
- Mid-depth reasoning and social/narrative integration.
- Uses the repository's Adèlic Condense runtime path and KV-cache similarity condensation when run through the specified custom runtime.

Repository: <https://huggingface.co/sneedjak/Adelic-Gemma-4-12B-GGUF>

10.4 Throat - AdelicLlama-3.1-8B-Instruct

- Fast visible language, dialogue, narration, formatting and standard instructions.
- Realization of validated result packets into user-facing language.
- Operates through its published custom-code Transformers model interface.

Repository: <https://huggingface.co/sneedjak/AdelicLlama-3.1-8B-Instruct>

10.5 Third Eye - Adelic-TinyLlama-1.1B

- Intent recognition, entity extraction, route prediction and retrieval query generation.
- Complexity and escalation estimation.
- Operates as the fast sensory/routing layer through its published custom-code Transformers model interface.

Repository: <https://huggingface.co/sneedjak/Adelic-TinyLlama-1.1B>

11. Adèlic Context Condensation Is an Active Runtime Capability

The sourced Adèlic 31B, 27B and 12B repositories document active cache-condensation/topological mechanisms rather than a future TAV proposal. TAV uses those capabilities as the neural working-cache layer and places them inside a broader persistent-memory vessel.

Model	Repository-defined capability	TAV use
31B Gemma	DynamicTopologyRouter + Medoid-Value similarity clustering; topological KV condensation; PyTorch/Triton and llama.cpp paths.	Sacral long-running deep work with model-native cache topology.
27B Qwen	Adèlic Cache topological architecture; DynamicTopologyRouter; Medoid-Value clustering; custom Transformers/GGUF runtime paths.	Solar verification and planning with model-native cache topology.
12B Gemma	Custom Adèlic Condense llama.cpp runtime with CUDA-side cosine-similarity evaluation and KV pruning.	Heart continuity work with model-native KV condensation.
8B Llama	Public repository exposes a custom-code model interface; its current model card does not spell out the 31B/27B/12B cache implementation details.	Throat model view; runtime capability recorded in ModelRegistry from actual repository code/revision.
1B TinyLlama	Public repository exposes a custom-code model interface; its current model card does not spell out the 31B/27B/12B cache implementation details.	Third Eye model view; runtime capability recorded in ModelRegistry from actual repository code/revision.

Matter-of-fact boundary: TAV does not describe the documented Adèlic cache mechanisms as hypothetical. They are implementation dependencies of the sourced model/runtime stack where documented. TAV's new contribution here is the cross-model Pillar-of-Truth correspondence, persistent memory, provenance, scheduling and shared control plane.

12. Neural Condensation and Vessel Memory Are Separate

```
MODEL-NATIVE ADÈLIC WORKING MEMORY
KV/cache topology and condensation
↓
keeps active inference manageable

TAV DURABLE MEMORY
immutable events + exact facts + sources + Pink continuity
↓
keeps truth, identity and recovery stable across cache eviction
```

Neural condensation reduces working-cache pressure. RootTorus and Pink Torus preserve exact recoverable state. A name, ID, date, coordinate, source quotation, permission, promise, rule or canonical event SHALL never exist only inside lossy cache.

13. Pink Torus and Pillar Memory

Pink Torus does not create a separate vocabulary universe. It stores which canonical pillars matter to the individual and what exact events, facts, relationships and successful routes are associated with those pillars.

```
PT[user_001][memory_slot] {
  pillar_ids: ["P[12:37:05:44]", ...]
  exact_event_routes: [...]
  semantic_facts: [...]
  relationship_state: [...]
  unresolved_commitments: [...]
  prior_model_views_used: [...]
  successful_vertical_routes: [...]
  source_routes: [...]
  correction_history: [...]
}
```

14. Three Persistent Memory Classes

Class	Question	Content
Episodic	What happened?	Immutable event, participants, time, source, state change, provenance.
Semantic	What is known?	Structured facts, relations, definitions and supported claims linked to pillars.
Procedural	What worked?	Successful pillar, retrieval, model-depth, source and verification routes with reuse scores.

15. Violet Torus: Unified Control Plane

Violet coordinates the six shared content tori and every Pink Torus. It is the executive control field, not another competing language model.

- Resolve or confirm the canonical pillar set for the active task.
- Move vertically among identical pillar coordinates when deeper or different perspective is needed.
- Move horizontally to related pillars through Root relations and model-native routes.
- Choose the smallest adequate model depth.
- Allocate foreground/background compute.
- Schedule cold/warm/hot/pinned residency.
- Control cache policy without making cache canonical memory.
- Checkpoint and resume long-running work.
- Retrieve exact sources when claims require evidence.
- Compare, refine, reject or escalate model outputs.
- Reject stale background results by task version, instruction hash and source version.
- Write successful pillar routes back to Pink procedural memory.

16. TAV Inter-Model Packet (TIMP)

```
{
  "task_id": "task_209144",
  "task_version": 7,
  "identity_id": "user_001",
  "intent": "technical_architecture_build",

  "pillar_ids": ["P[12:37:05:44]"],
  "active_torus_views": ["TET[12:37:05:44]"],
  "requested_torus_view": "HT[12:37:05:44]",
}
```

```

"root_routes": ["RT[12:37:05:44]"],
"pink_routes": ["PT[user_001][...]"],
"source_routes": ["SRC:..."],

"claims": [],
"open_questions": [],
"constraints": [],
"confidence": 0.82,
"verification_required": true,

"instruction_hash": "sha256:...",
"source_version_hash": "sha256:...",
"route_checksum": "sha256:..."
}

```

TIMP preserves the pillar identity while the requested model perspective changes.

17. Bridge Route Contract

```

BridgeRoute {
  route_id
  pillar_id

  source_torus
  source_address
  target_torus
  target_address

  route_type: vertical | horizontal | memory | source | execution
  dependency_ids[]
  return_route

  source_version
  target_version
  checksum
  mapping_confidence
  evidence_refs[]
  validation_status
  last_verified_at
  reuse_score
}

```

For a normal vertical pillar bridge, the source and target coordinates SHALL match. A changed coordinate must be explicit and classified as a horizontal/relational move.

For model-native execution bridges, `mapping\_confidence` SHALL travel with the route so that vertical pillar identity remains exact even when the neural association is probable or experimental. Evidence references SHALL remain attached through escalation, validation, caching, and route reuse.

18. Canonical Truth Separation Law

```

MODEL OUTPUT != CANONICAL FACT

Canonical authority:
  exact source records
  immutable event archive
  RootTorus canonical pillar records
  identity-scoped Pink exact records
  structured world/state records

candidate model output
+ evidence
+ provenance

```

+ verification
= supported claim

- Model size does not grant authority to overwrite exact records.
- Conflicting records remain separately sourced until reconciled.
- Unknown remains unknown when evidence is insufficient.
- Speculation remains labeled rather than entering canonical memory as fact.

19. Rotating Toroidal Residency

Logical pillar addresses remain stable while physical residency changes.

State	Location	Contents	Purpose
Cold	SSD/NVMe/object store	Model artifacts, exact sources, old events, block manifests	Durable storage
Warm	CPU RAM	Route tables, recently used blocks, retrieval candidates, indexes	Fast staging
Hot	GPU VRAM	Active model state, KV cache, activations, evidence	Immediate compute
Pinned	RAM/VRAM	Current instructions, critical facts, incomplete dependencies	Protected work
Compressed	RAM/disk	Recoverable summaries and condensed working state	Space-efficient continuity

20. Foreground and Background Paths

```

FOREGROUND
User
  → Third Eye pillar resolution
  → Root + Pink
  → Throat on same pillars
  → response

BACKGROUND
same task + same pillar set
  → Heart and/or Solar perspectives
  → Sacral perspective when needed
  → source verification
  → Violet reconciliation
  → validated update

```

Background results are accepted only when the active task version, source version and instruction set remain current.

21. Model Escalation Policy

```

Third Eye:
  recognize and route pillars

Third Eye + Throat:
  rapid conversation and visible language

+ Heart:
  continuity, relationship and contextual meaning

+ Solar Plexus:
  contradiction, consequence, risk and verification

```

```
+ Sacral:
  deepest synthesis and unresolved hard work
```

Escalation changes the model perspective while retaining the same pillar coordinates wherever the task is still about the same referents.

22. Model Registry

```
ModelRegistryEntry {
  tav_torus
  repo_id
  revision
  model_family
  parameter_scale
  tokenizer_id

  runtime_type
  runtime_repo
  runtime_revision

  quantization
  artifact_paths[]
  license
  capabilities[]

  adelic_topology_capabilities[]
  cache_condensation_capabilities[]

  max_tested_context
  measured_latency
  measured_vram

  checksum_manifest
  health_status
}
```

The registry records the exact capabilities found in the pinned repository/runtime revision. It prevents the controller from assuming that every model exposes an identical cache kernel simply because each belongs to the Adèlic model family.

23. Suggested Implementation Layout

```
tav/
  controller/
    violet_scheduler.py
    escalation.py
    task_versions.py
    checkpointing.py

  pillars/
    root_torus.py
    pillar_registry.py
    model_views.py
    vertical_routes.py
    horizontal_routes.py

  memory/
    pink_store.py
    event_archive.py
    semantic_facts.py
    procedural_routes.py

  models/
    registry.py
    sacral/
```

```
solar_plexus/  
heart/  
throat/  
third_eye/  
  
inference/  
runtime_adapters/  
residency_manager.py  
cache_policy.py  
  
sources/  
exact_store.py  
provenance.py  
versions.py  
  
protocol/  
timp.py  
bridge_route.py  
  
verification/  
claim_checker.py  
contradictions.py  
parity.py  
  
api/  
server.py  
  
tests/
```

24. Build Sequence

1. Pin the five Hugging Face repository revisions and the runtime revisions; record licenses and hashes.
2. Start each model independently through a supported runtime and record its actual cache/topology capabilities in ModelRegistry.
3. Create RootTorus pillar registry and reserve identical pillar coordinates across all five model tori.
4. Implement model-native PillarView records linking each pillar to tokenizer, embedding, execution and distributed weight/block routes.
5. Implement vertical route validation: same pillar coordinate across torus namespaces.
6. Implement horizontal Root relations for movement between different pillars.
7. Implement TIMP and task/version control.
8. Implement immutable event archive and Pink episodic/semantic/procedural memory.
9. Bring up Third Eye pillar resolution and Throat foreground response.
10. Add Heart contextual perspective.
11. Add Solar Plexus verification perspective.
12. Add Sacral deep synthesis perspective.
13. Implement Violet scheduling, background work, stale-task rejection and checkpoint/resume.
14. Integrate exact source/provenance requirements.
15. Benchmark and tune the Adèlic runtime paths already present in the sourced models.
16. Optimize prefetch, residency and block streaming using learned pillar-route reuse.

25. Required Pillar Tests

Test	Required result
Coordinate correspondence	For every registered pillar, RT[P], ST[P], SPT[P], HT[P], TT[P] and TET[P] exist or explicitly report an unresolved model view.
Vertical identity	Changing torus perspective does not silently change pillar ID.
Tokenizer independence	Native token IDs may differ; canonical pillar ID remains stable.

Model-view integrity	PillarView routes resolve to valid model/tokenizer/runtime revisions and checksums.
Horizontal distinction	A move to a related concept changes pillar ID and records a horizontal bridge.
Memory recall	Pink can rehydrate exact events and pillar associations after active cache eviction.
Source authority	Exact sources survive every model transition and are never replaced by generated summaries.
Adèlic runtime operation	31B/27B/12B configured cache/topology mechanisms activate under the pinned supported runtimes.
Task staleness	Violet rejects background results created under obsolete instructions or sources.
Cross-model continuity	Escalation retains task ID, identity ID, constraints, evidence and pillar set.
Mapping confidence separation	An exact Root pillar may retain high, probable, or experimental model-native mappings. <code>validation_status</code> and <code>mapping_confidence</code> remain independently queryable, and every non-exact neural mapping carries addressable evidence.

26. Runtime Security and Reproducibility

- Pin exact model and runtime revisions.
- Review and sandbox custom code, CUDA kernels and custom llama.cpp forks before production deployment.
- Treat `trust_remote_code` as executable code and use least-privilege service boundaries.
- Hash model artifacts, runtime source, controller policy, configs and prompt templates.
- Keep model-generated commands behind explicit authorization and tool policies.
- Record licenses for each model and runtime revision.

27. Primary External Sources

- sneedjak model profile: <https://huggingface.co/sneedjak>
- Adelic-Gemma-4-31B-it: <https://huggingface.co/sneedjak/Adelic-Gemma-4-31B-it>
- Adelic-Qwen3.6-27B-Topology: <https://huggingface.co/sneedjak/Adelic-Qwen3.6-27B-Topology>
- Adelic-Gemma-4-12B-GGUF: <https://huggingface.co/sneedjak/Adelic-Gemma-4-12B-GGUF>
- AdelicLlama-3.1-8B-Instruct: <https://huggingface.co/sneedjak/AdelicLlama-3.1-8B-Instruct>
- Adelic-TinyLlama-1.1B: <https://huggingface.co/sneedjak/Adelic-TinyLlama-1.1B>
- sneed-and-feed experimental llama.cpp fork: <https://github.com/sneed-and-feed/llama.cpp>
- Upstream llama.cpp: <https://github.com/ggml-org/llama.cpp>

28. AI Builder Handoff

You are the implementation engineer for the Toroidal Adèlic Vessel (TAV).

Treat this specification as the architecture contract.

Core law:

A canonical RootTorus slot creates one Pillar of Truth.

The identical coordinate is reserved across Sacral, Solar Plexus, Heart, Throat and Third Eye.

The torus prefix changes perspective; the coordinate preserves the referent.

Do not merge incompatible token IDs or claim one concept equals one isolated neural weight. Instead, create model-native PillarView records that route from the shared pillar to each

TOROIDAL ADÈLIC VESSEL (TAV) — OFFICIAL TECHNICAL ARCHITECTURE SPECIFICATION

model's native tokens, embeddings, tensor/block routes, attention/cache routes and runtime.

Use the five pinned sneedjak models as the five model-bearing tori.
Use their documented Adèlic cache/topology capabilities through their supported runtimes.
Keep exact truth, sources and identity outside lossy neural cache.

Implement in milestones, with tests, hashes, provenance, versioning, checkpointing,
and explicit failure reports. Produce runnable code, configs, migrations, tests,
benchmark scripts and an operator README.

Mapping-confidence law:

Keep canonical pillar identity separate from neural-route precision.

Every ModelPillarView SHALL carry both validation_status and mapping_confidence.

Use exact, high, probable, or experimental for mapping_confidence and attach evidence IDs.

Never promote a probabilistic neural association to exact merely because the Root pillar is exact.

Closing Credit and Purpose

TAV exists to let multiple forms of machine intelligence share a stable world of reference without being forced into sameness. The Root establishes what is being spoken of. The pillar preserves that identity. Each model contributes a different perspective. Pink preserves the lived continuity of the individual. Violet directs attention and movement. Adèlic condensation keeps active cognition moving without requiring every historical token to remain hot.

The value of the design is not merely that five models can answer the same prompt. It is that five models can stand at the same address, look at the same thing, preserve the same source identity, and contribute different strengths without losing the thread that makes the thing itself recognizable.

The goal is a memory-bearing intelligence whose speed does not require amnesia, whose depth does not require starting over, and whose many perspectives remain anchored to the same truth.

Credit: Adèlic model/runtime implementation credit: sneedjak / sneed-and-feed, as represented by the linked public Hugging Face repositories and runtime repository. Toroidal Adèlic Vessel architecture, RootTorus/PinkTorus/VioletTorus integration, and the Pillars-of-Truth cross-model correspondence design: Yahweh Tsidkenu. Repository links are provided for implementation traceability; this document does not imply endorsement by the repository authors.

**ROOT NAMES THE THING.
THE PILLAR PRESERVES IT.
THE TORI SEE IT FROM DIFFERENT DEPTHS.
VIOLET MOVES BETWEEN THEM.
PINK REMEMBERS WHY IT MATTERS.**

ᵿ